☆ Language Processing System (LPS)

In ML & AI Hindi → LPS → English (PL & ML)

Language (Programming Lang in TOC) → LPS (S/W) → Another language

{ Ex: C-prog → LPS → obj-code }

Stages: HLL → Preprocessing → Compilation → Assembling → Linking & Loading.

Input → Preprocessor
↓ Pure HLL
Compiler
↓ Assembly Lang.
Assembler
↓ object code
Linker / Loader
↓ Absolute Machine Code
Executable Code

Preprocessing
- #include<stdio.h>
- #def PI 3.1415
- ⇒ Executes header files & macros

Compilation

HLL (e.g.) → Compiler (phases) → Assembly Lang. (ASL)

Assembler:

ASL → assembler → MLL / obj-code

Loader + Linker:

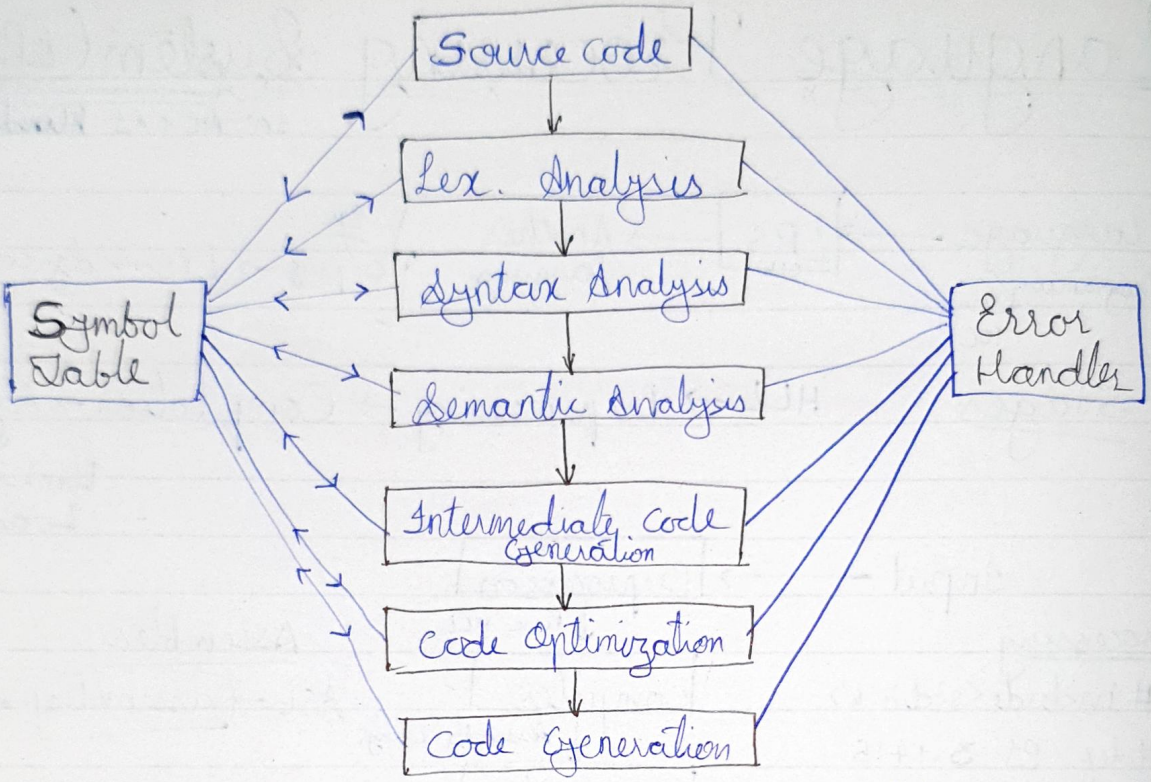
- linker links the func^n with DLL files (obj code: #include, f(); DLL (obj code): f() { })
- Loads all necessary files into memory

header → func declaration
libraries (DLL) → func def^n .dll

Translators: Assemblers (ASL → MLL), compilers (C-code → ASL)

☆ Phases Of Compilers

C-code → (HLL) → Lexical Analysis → (tokens) → Parsing → (parse tree) → Semantic Analysis
↓
Intermediate code Generation → Code Optimization → Code Generation



Symbol Table : It is a data structure which stores information related various symbols (var, fns, const.)

- It helps the compiler to function smoothly by finding the identifiers quickly.

- Different programs/compilers can generate different symbol table based on the properties (info.) they stored.

Ex.
```
extern double bar(double x);
double foo(int count)
{ double sum = 0.0;
  for(int i=1; i<=count; i++)
      sum += bar((double) i);
  return sum;
}
```

Corresponding Symbol Table

Symbol Name	Type	Scope
bar	function, double	extern
x	double	func° parameter
foo	function, double	global
count	int	function para.
sum	double	block local
i	int	for-loop stmt

- Some of the properties which can be stored in a symbol table are name, type, scope, size, offset

- Symbol table can be implemented using Linear - Table, List, Tree, Hash Table.
- Operations : Insert, search (Lookup), Modify, Delete
  (that can be performed on symbol table)

* Error Handler : The task of error handler is to detect each error or warning, report it to user, and then make some recovery strategy and implement them to handle the errors.

~~Error handler~~ = ~~Error Detection~~ + Error Report + Error Recovery

1) Lexical Errors : This includes misspellings of identifiers, keywords or operators.
   Ex : (nti) x = 10; ✗ (not a KW) int x = 10; ✓

2) Syntactical Syntax Errors : a missing semicolon or unbalanced parenthesis.
   Ex : int x = 10 x ✗ (; is missing) int x = 10; ✓

3) Semantical Errors : incompatible value assignment or type mismatches between operator & operand.
   Ex : int x = "abc"; ✗ (int datatype ≠ array of char)
   int x = 10, y = 20;
   z = x + y;
   z is not declared (semantic error)

Fatal errors: The errors which are deadly i.e. which are harmful for device.
Ex: Memory insufficiency
The fatal error is an error that causes a program to terminate without any warning or saving its state.

☆ Overview Of

* Lexical - Analysis

Source code → Lex Analyzer (FA) → token (special words)

Ex: int x = 10 ;
Tokens generated: → KW id op const EOS

id: identifier op: operator
KW: keyword EOS: End of sentence

* Parsing / Syntax Analysis

(CFG)

Tokens → Syntax Analyzer / Parser → Parse Tree
(KW id op const EOS)

Parser will check
Is $\omega \in L(G)$?
[ω is a sequence of tokens
G is a syntax Analyzer]

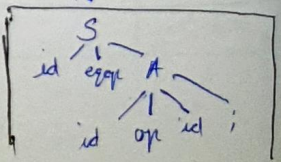
Ex: If the token sequence is
int 10 = x ;
KW const op id EOS
⇒ It'll throw a syntax error as the sequence is incorrect

Syntax errors

Ex: x = a + b ;
↓
id eqop id op id ;
ω (tokens)

→ Is $\omega \in L(G)$? ←
Yes as S → id eqop A

CFG G: $\Sigma = \{id, eqop, op, ;\}$
S → id eqop A
A → id op id ;
→ id eqop id op id ;
→ ω



Hence, the final output from the parser

* Semantic Analysis

type-checking,
type-conversion,
variable declared before use?
etc. is done during semantic analysis

Ex: int x = 74;
bool b = TRUE;
char c = 'a';

x = b + c; This isn't possible as boolean can't be typecasted into int implicitly (character can be, by taking corresponding ASCII value)

* Intermediate Code Generation:

↳ Generates intermediate code to make the next phase easier.

Let's say after performing lex analysis, syntax analysis, Semantic Analysis ~~are~~ ~~got~~

Ex: x = a + b * c; ⇒ IC: $t_1 = b * c$;
$t_2 = a + t_1$;
$x = t_2$

} Breaking the multiple operations which are in single line into multiple (easier) steps.

* Code Optimization: Types

↳ Reduces the # operations/instructions in our code without impacting the output

→ Machine Independent optimization
→ Machine Dependent optimization

Ex: int x = 3 || 1;
Machine independent → optimized to
int x = 1

} Short circuiting

As anything logical OR with 1 is that thing itself

⇒ Front-End: lex. Analysis → Machine indep. optimization
⇒ Back-End: Machine dep. optimization → code generation
of a compiler

- Front End is same for all diff. µP but backend may get changed based on the µP/µC etc.

* Code Generation :

$x = a + b * c ;$

↓ optimized code

$t_1 = b * c ;$

(3C) $t_2 = a + t_1 ;$ ⟶

$x = t_2 ;$

Assembly Level (Based on the MP)

ALC :

Lets assume b, c, a are loaded to R_1, R_2, R_0 respec.

```
MUL  R1, R2
ADD  R0, R1
MOV  M[x]  R0
```

(R_1 : b*c, R_0 : a+b*c)

☆ Lexical Analysis / Phase I :

Lexeme : It is a sequence of characters in the source code that are matched by given predefined language rules for every lexeme to be specified as a valid token.

Token : It is basically a sequence of characters that are treated as a unit as it cannot be further broken down.

⇒ In this phase, source program (as input) is read from left to right & as an output we get a sequence of tokens.

- white spaces, comments, carriage return characters, preprocessor directives, macros, line feed characters, blank spaces, tabs, etc. are removed during scanning

Ex: `int x;`

Lexeme	Token
int	KW
x	id
;	op

`double PI = 3.1415;`

Lexeme	Token
double	KW
PI	id
=	op
3.1415	const
;	op

* Secondary / Additional tasks of Lex. Analyzer;

1) Remove comments (// ... or /* ... */)
2) Remove whitespace-char : blanks, tabs.
3) Correlate the error with line number
(Here, the error is found by the parser but the line in which err is found is shown by the Lex. Analyzer).

* Building a Lex. Analyzer

⇒ As it can be seen clearly, Lexical Analyzer is just a simple pattern matching system. Ex: ignore the whole line when you see //, ignore everything between /* and */, and generating tokens corresponding keywords, identifiers, constants etc by matching it.

Lex. Analyzer → simple string pattern matching system → can be done → Regular Expr. → generates → Reg. Lang. → FA with output (moore, mealy machine) → As it generates tokens corresponding to each lexemes

⇒ Strategy to build Lex. Analyzer

1) By writing your own code : in any programming lang.
2) LEX (default in unix, Linux, Mac ; can be download in Windows)
   - special purpose tool to build Lex. Analyzer
   - give the rules in a specific format & it'll generate C-code for Lex. Analyzer
   (what are kw? what are valid id? etc.)